

Step 2 — One Tool, One Loop: the Smallest Possible Working Agent — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain), **Advanced** (what a loop that Step 3 grows into a real Worker needs to already get right). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

An "agent," at its simplest, is a model that can decide to use a tool instead of just answering directly. This step gives it exactly one tool — a plain Python function, decorated so LangChain knows how to offer it to the model — and wraps a small loop around the model call: ask the model, check if it wants to use the tool, if so run the tool and tell the model what happened, then ask again.

Pick the simplest possible tool for this — something with no external dependency, like a calculator function. You want the *loop* to be the thing you're proving works, not the tool itself. Deliberately don't reach for `AgentExecutor` here — building the loop by hand once is the whole point of this step, per Doc07.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Two things to build:

```
# tools.py
@tool(args_schema=CalculatorArgs)
def calculator(...) -> str:
```

```
# agent.py
def run_agent(task: str) -> AgentResult:
```

The tool needs a Pydantic argument shape (`CalculatorArgs` , with typed fields like `expression: str`) so LangChain can validate what the model sends it — this is the same discipline Doc06 teaches: never trust the model's raw arguments without a schema checking them first.

`run_agent()` 's loop needs the model bound to the tool list (`model.bind_tools([calculator])`), so the model's response can include a `tool_calls` field. Check that field each time: if it's present, actually run the named tool with the given arguments, add the result back into the message history as a tool message, and call the model again. If it's empty, the model gave a final answer — stop the loop.

Sketch the loop's shape (ask → check for `tool_calls` → run tool or stop) before writing the tool or the full loop body.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The exercise's own loop has no exit condition beyond "the model eventually stops asking for the tool" — fine here, since a single reliable tool naturally finishes in 1-2 rounds. But this exact loop shape is what Step 3 grows into the real Worker, hard limit and all. Two things worth getting right now, so Step 3 is a true extension and not a rewrite.

First, what actually happens if `calculator.invoke(call["args"])` raises — say the model sends an expression that isn't valid Python. An un-caught exception here crashes the whole agent mid-loop, which is a much worse failure than the model just getting a clear error message and trying again. Catch it, and feed the error back as the tool's result (still a `ToolMessage`), the same shape as a successful result — the model can often self-correct from a labeled error, but it never even gets the chance if your code crashes first.

Second, `steps: list[str]` as plain strings is fine for printing, but throws away structure — you can't later filter "just the tool calls" or count how many rounds actually ran without re-parsing text. A small `AgentStep` model (tool name, args, result, whether it errored) keeps the same information queryable.

The extra pieces: - A `try/except` around the tool call, catching the general case (`Exception`) for this exercise, returning `f"Error: {e}"` as the `ToolMessage` content instead of letting it propagate. - An `AgentStep(BaseModel)` with fields like `tool: str`, `args: dict`, `result: str`, `ok: bool`, used instead of a plain string in `AgentResult.steps`.

Sketch how a failing calculator call should look in the message history before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume the tool call succeeds every time — reasonable for a first working loop. Advanced treats failure as a normal thing this loop needs to handle even with the "friendliest" possible tool, because the loop's actual shape (not its exact tools) is what Step 3 reuses wholesale — a failure path added now is a failure path Step 3 doesn't have to invent under more pressure, with more tools in play.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a calculator tool: takes an expression, returns the computed result

function run_agent(task):
    messages = [system message, task as a human message]
    loop:
        ask the model, with the calculator tool available
        if the model asked to use the calculator:
            actually run it
            add the result to messages as a tool result
            ask again (loop continues)
        else:
            this is the final answer, stop and return it
```

Here's almost the whole loop — try finishing the rest yourself:

```
def run_agent(task):
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), HumanMess

    while True:
        response = model.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return response.content
        for call in response.tool_calls:
            result = calculator.invoke(call["args"])
            messages.append(ToolMessage(content=result, tool_call_id=call["id"]))
```

Expected output, run against "What is 47 * 6?" : the loop makes exactly one tool call round, then returns a final answer mentioning 282 . Run against "What's the capital of Japan?" and confirm zero tool calls happen — the model should answer directly.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
tools.py:
class CalculatorArgs(BaseModel):
    expression: str

    @tool(args_schema=CalculatorArgs)
    def calculator(expression: str) -> str:
        return str(eval(expression))    # fine for this exercise; a real project w

agent.py:
class AgentResult(BaseModel):
    answer: str
    steps: list[str]

def run_agent(task: str) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), Human
    steps = []

    while True:
        response = model.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return AgentResult(answer=response.content, steps=steps)

        for call in response.tool_calls:
            result = calculator.invoke(call["args"])
            steps.append(f"called {call['name']} with {call['args']} -> {resu
            messages.append(ToolMessage(content=result, tool_call_id=call["id
```

Notice the loop has no exit limit yet — that's deliberate, it's Step 3's job. Write `main.py` printing each think/act/observe step from `AgentResult.steps`, and the two-prompt test (one that clearly needs the calculator, one that doesn't), before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class AgentStep(BaseModel): tool, args, result, ok

function run_agent(task) -> AgentResult:
    ... same loop as Intermediate ...
    for each call in response.tool_calls:
        try:
            result = run the tool
            record an AgentStep with ok=True
        except Exception as e:
            result = "Error: " + str(e)
            record an AgentStep with ok=False
        add result to messages as a ToolMessage either way
```

Here's almost the whole thing — fill in the missing piece yourself:

```
class AgentStep(BaseModel):
    tool: str
    args: dict
    result: str
    ok: bool

def run_agent(task: str) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator])
    messages = [SystemMessage("Use the calculator tool for any math."), HumanMess
    steps: list[AgentStep] = []

    while True:
        response = model.invoke(messages)
        messages.append(response)

        if not response.tool_calls:
            return AgentResult(answer=response.content, steps=steps)

        for call in response.tool_calls:
            # your turn: wrap this in try/except, build an AgentStep either way,
            # and append it to `steps` before adding the ToolMessage
            ...
```

Fill in the try/except and `AgentStep` construction yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the loop's core shape (ask → check `tool_calls` → run or stop) never changes across all 3 levels — Basic proves it runs, Intermediate adds real types and a structured `AgentResult`, Advanced adds the one thing that only shows up once something actually goes wrong: a tool call that raises instead of returning cleanly. That failure path is small here (this tool barely ever fails), but it's the exact seam Step 3 plugs a genuinely-failing tool into.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)